In this example, the console logger explicitly restricts logs written in the `Microsoft` namespace to `Warning` or above, so the console logger ignores the log message shown. Conversely, the file logger doesn't have a rule that explicitly restricts the `Microsoft` namespace, so it uses the configured minimum level of `Information` and writes the log to the output.

TIP Only a single rule is chosen when deciding whether a log message should be written; rules aren't combined. In figure 26.8, rule 1 is considered to be more specific than rule 5, so the log is written to the file provider, even though technically, both rules could apply.

You typically define your app's set of logging rules using the layered configuration approach discussed in chapter 10, because this lets you easily have different rules when running in development and production.

TIP As you saw in chapter 11, you can load configuration settings from multiple sources, like JSON files and environment variables, and can load them conditionally based on the `IHostingEnvironment`. A common practice is to include logging settings for your production environment in appsettings.json and overrides for your local development environment in appsettings .Development.json.

`WebApplicationBuilder` automatically loads configuration rules from the `"Logging"` section of the `IConfiguration` object. This happens automatically, and you rarely need to customize it, but listing 26.6 shows how you could also add configuration rules from the `"LoggingRules"` section using `AddConfiguration()`.

NOTE `WebApplicationBuilder` always adds the configuration to load from the `"Logging"` section; you can't remove this. For this reason,